

PodBeater: Exploiting Multi-Value Affinity for Efficient Co-Location Attacks in Kubernetes

Yiju Jung^[0009-0009-3068-0178], Hyerean Jang^[0000-0003-4100-9338], and
Youngjoo Shin^{[0000-0003-4831-7392]*}

School of Cybersecurity, Korea University, Seoul, 02841, Republic of Korea
{wjddlwn2637, hr_jang, syoungjoo}@korea.ac.kr

Abstract. Kubernetes supports multi-tenancy by allowing multiple users to share physical infrastructure. However, this design leads to co-location attacks, where an attacker deploys attack pods onto the same node as victim pods to exploit shared hardware resources. While previous work demonstrates the feasibility of such attacks, it assumes that the attacker has sufficient knowledge of the victim pod’s configuration. This paper presents PodBeater, a novel co-location attack that exploits Kubernetes’s multi-value affinity feature to steer the scheduler’s pod placement decisions. Unlike previous work, PodBeater is designed to operate even under a realistic threat model where the attacker lacks knowledge of the victim pod’s configuration. We evaluate PodBeater on both a small-scale cluster with 20 nodes and 350 pods and a large-scale cluster with 40 nodes and 800 pods, demonstrating its effectiveness under both previous and more realistic threat models. Our results show that PodBeater achieves co-location with fewer attack pods compared to previous works.

Keywords: Kubernetes · Co-location attack · Affinity mechanism

1 Introduction

Kubernetes [7] has become the de facto standard for container orchestration in modern cloud infrastructures. A core component of the Kubernetes architecture is the scheduler, which assigns pods to suitable nodes based on resource availability and user-specified scheduling requirements. While this scheduling mechanism offers flexibility and efficiency, it also introduces new security challenges [5]. Moreover, Kubernetes supports multi-tenancy by allowing multiple users to share the same physical infrastructure. This design leads to co-location attacks [1, 10, 15], where an attacker deliberately deploys attack pods on the same node as target pods. Once co-located, the attacker can exploit shared hardware resources to mount microarchitectural side-channel [9] or denial-of-service attacks [14].

Previous work, REPTTACK [5], demonstrates the feasibility of co-location attacks in Kubernetes cluster. Specifically, it exploits the Kubernetes scheduler by allowing an attacker to influence pod placement decisions through carefully

* Corresponding author: syoungjoo@korea.ac.kr

crafted user-specified scheduling requirements. By replicating the victim pod’s scheduling requirements, the attacker can deliberately steer the scheduler to co-locate attack pods on the same node as the victim pod.

However, REPTTACK has two main limitations. First, it assumes a threat model in which the attacker possesses sufficient knowledge of the victim pod’s scheduling requirements, which is strong in practical settings. This assumption does not reflect the real-world Kubernetes cluster, as such information is typically not exposed to unprivileged users. Second, previous work does not account for additional scheduling requirements, such as multi-value affinity conditions, which can significantly influence the scheduler’s placement decisions.

To overcome these limitations, we improve the previous work by presenting *PodBeater*, a co-location attack exploiting multi-value affinity. In Kubernetes, nodes and pods are labeled with user-specified attributes (e.g., location or hardware type), which the scheduler uses to identify nodes that meet a pod’s scheduling requirements. Multi-value affinity enables a pod to specify multiple values for a given label, expanding the set of candidate nodes. As a result, the scheduler considers more nodes for placement, increasing the probability that a pod is scheduled on the same node as the victim. PodBeater exploits this scheduling behavior to achieve co-location with the victim pod using fewer attack pods.

We evaluate PodBeater under two threat models: (1) $\mathcal{T}_u$, which is the same as the previous work [5] where the attacker has sufficient knowledge of the victim pod’s scheduling requirements and (2) $\mathcal{T}_r$, where the attacker lacks knowledge of the victim pod’s scheduling requirements, more realistic than $\mathcal{T}_u$. We design multiple attack strategies and assess them on both small- and large-scale clusters. We evaluate our work under $\mathcal{T}_u$ to compare it with REPTTACK. Under $\mathcal{T}_r$, we analyze PodBeater’s effectiveness using both single-value and multi-value affinity condition. In our experiment, PodBeater consistently achieves co-location with fewer attack pods than the previous work and single-value affinity strategies, demonstrating superior efficiency in reducing attack cost. Finally, we discuss several mitigations against the proposed attack.

Contributions. Our main contributions are as follows:

- We present PodBeater, a co-location attack that leverages multi-value affinity in the scheduling process of Kubernetes.
- We evaluate PodBeater under various threat models, demonstrating that it achieves co-location with fewer attack pods compared to the previous attack.
- We perform large-scale experiments on a 40-node Kubernetes cluster with 800 pods to quantitatively evaluate how different attack strategies affect the number of attack pods required to achieve co-location.

2 Related Work

Co-Location attack in Kubernetes. Previous work [5] introduces a co-location attack that exploits the scheduling behavior of Kubernetes. It assumes the attacker has sufficient knowledge of the victim pod’s scheduling requirements. By replicating these requirements, the attacker achieves co-location with the victim

pod by deploying a small number of attack pods. In contrast, our work explores multi-value affinity to enable more efficient co-location attacks in Kubernetes, and demonstrates that PodBeater outperforms REPTTACK. We further evaluate PodBeater under realistic threat models, and propose diverse attack strategies to quantitatively assess its effectiveness.

Co-Location attacks beyond Kubernetes. Co-location attacks have been studied in various cloud environments. Zhao et al. [15] exploit the load balancer of Google Cloud Run to spread attack instances across multiple physical machines. Makrani et al. [10] used fake traces that mimic the victim’s behavior to induce co-location. Atya et al. [1] demonstrated brute-force methods in Amazon EC2 by repeatedly launching VMs until co-location is achieved. Other studies analyzed how placement policies affect attack performance. Han et al. [6] showed that attackers can improve the success rate of co-location by adjusting the number of VM launches based on allocation policies. Varadarajan et al. [13] showed that manipulating deployment parameters such as launch timing and instance count can improve the success rate of co-location in public cloud.

Affinity policy in cloud. Affinity policy has also been employed in the cloud to improve placement decisions and optimize system performance. Pachorkar et al. [12] and Chen et al. [2] both proposed an affinity-aware placement mechanisms to optimize resource usage and reduce network overhead. Moreno-Vozmediano et al. [11] proposed a new affinity-based placement policy to improve orchestration mechanisms for deploying high-availability services.

Unlike previous work, our work shows that attackers can influence scheduling decisions by exploiting affinity-based configurations. While affinity policies are designed to optimize resource usage and service availability, our findings reveal that they can be exploited to perform co-location attacks.

3 Background

3.1 Kubernetes orchestration mechanism

Kubernetes is an orchestration platform that automates the deployment and management of containerized applications in the cloud. A Kubernetes cluster consists of *nodes* (i.e., physical or virtual machines) that provide resources to run *pods*, the smallest scheduling units that encapsulate one or more containers.

Kubernetes provides an efficient scheduling mechanism to deploy pods onto appropriate nodes within the cluster. Users can specify scheduling requirements by attaching labels (i.e., user-specified key-value pairs), which are expressed as affinity conditions. These requirements, defined in the pod specification, influence the scheduler’s decision to select a suitable node.

Overview of Kubernetes scheduler. The Kubernetes scheduler determines pod placement based on the current state of cluster resources and user-specified scheduling requirements. As shown in Fig 1, users specify these scheduling requirements using labels within the pod specification. The scheduling process comprises two phases. In the filtering phase, the scheduler excludes nodes that

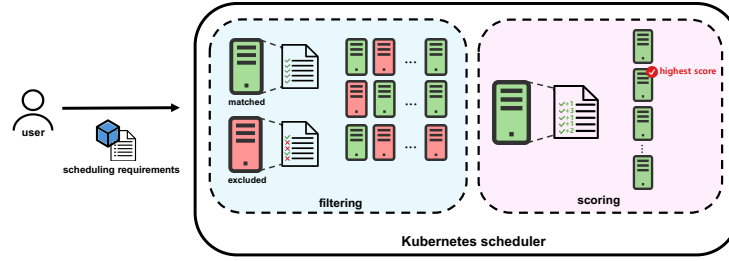


Fig. 1. Workflow of Kubernetes scheduler to determine a suitable node.

fail to meet the pod’s resource requirements. In the scoring phase, it evaluates the remaining candidates and assigns scores. The scheduler then selects the node with the highest score for pod placement.

Affinity scheduling mechanism in Kubernetes. Kubernetes provides an affinity mechanism to reflect user-specified scheduling requirements. Affinity allows users to define placement conditions that constrain a pod to run on a specific node or be co-located with, or explicitly separated from, other pods. There are three types of affinity: *nodeAffinity*, *podAffinity*, and *podAntiAffinity*. *nodeAffinity* restricts a pod to nodes with specific labels. *podAffinity* places a pod on nodes already hosting pods with matching labels. *podAntiAffinity* ensures separation from pods with specified labels, often used to improve availability or reduce resource contention. Users define these conditions in the pod specification using key-value labels.

Affinity conditions are evaluated during both the filtering and scoring phases of the scheduling process, directly influencing pod placement decisions. User can specify affinity conditions with either a single value or multiple values for a given key. The condition is satisfied if any of the specified values match the corresponding label on the node, making it eligible for scheduling. For example, if a pod is expected to be placed on nodes labeled `disktype=ssd`, this requirement can be specified using *nodeAffinity*. Similarly, if the *podAffinity* specifies the key `color` with a single value `red`, the pod will be scheduled on a node that already hosts another pod labeled `color=red`. When multiple values are specified for the key `color`, such as `values: [red, orange, yellow]`, then any node hosting a pod labeled one of these values (e.g., `color=orange`) satisfies the condition.

3.2 Co-Location attack

Orchestration systems such as Kubernetes operate in multi-tenant environments, allowing pods from different users to share the same nodes. Co-location attack exploits this environmental characteristic by intentionally placing the attack pod on the same node as the victim pod, forcing them to share physical resources.

Once co-located, the attacker can exploit shared hardware to perform various threats, such as cache-based side-channel or denial-of-service (DoS) attacks, potentially leaking information or degrading service availability. Since co-location enables these threats, it is a significant security concern.

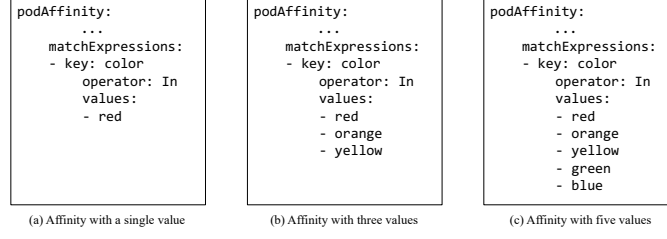


Fig. 2. Comparison of single-value and multi-value affinity.

4 PodBeater attack

In this section, we propose PodBeater, a co-location attack that exploits Kubernetes’s multi-value affinity to reduce the attack cost by minimizing the number of pods required to achieve co-location with a victim pod. In Section 4.1, we present the motivation behind PodBeater. In Section 4.2, we describe the attack procedure for conducting PodBeater in Kubernetes.

4.1 Motivation

Previous work demonstrates that Kubernetes’s affinity-based scheduling mechanism can be exploited to co-locate with a victim pod on the same node. However, it assumes that the attacker has sufficient knowledge of the victim pod’s scheduling requirements and considers only a single strategy with fixed affinity conditions. In real-world cloud environments, attackers typically lack such knowledge, and affinity conditions are often configured with multiple values.

Therefore, it is necessary to evaluate attack strategies under more realistic threat models and to explore attack strategies that leverage multi-value affinity configurations. Unlike previous work, we consider a more realistic threat model in which the attacker lacks such knowledge. Moreover, since Kubernetes supports multi-value affinity, we leverage this feature to design various attack strategies that achieve co-location with fewer attack pods than both previous work and single-value affinity strategies. To this end, we introduce PodBeater, a co-location attack that employs multi-value affinity strategies and evaluates their effectiveness under both $\mathcal{T}_r$ and $\mathcal{T}_u$.

4.2 Attack procedure

PodBeater exploits Kubernetes’s multi-value affinity, which influences the scheduler’s placement decision to select a suitable node. The attacker performs the attack through the following steps.

- Step 1** The attacker defines a pod specification with an affinity condition that includes N affinity values to steer the scheduler’s placement decision.
- Step 2** The attacker submits the pod specification, and the scheduler deploys the attack pod to a suitable node based on the provided affinity values.

Table 1. Kubernetes cluster configurations.

	# of master nodes	# of worker nodes	# of pods
Small-scale cluster	1	20	350
Large-scale cluster	1	40	400

Step 3 The attacker verifies whether the attack pod is co-located with the victim pod on the same node.

Step 4 Steps 1-3 are repeated until the attack pod is successfully co-located with the victim pod.

Since affinity feature influences the scheduler’s placement decision to assign a pod to a suitable node, the performance of PodBeater can vary depending on the number of affinity values N specified in the pod specification. Fig 2(a) illustrates an example of the single-value affinity, where the affinity condition specifies only one value. Fig 2(b), and Fig 2(c) illustrate multi-value affinity, where the affinity condition specifies three and five values, respectively. Whenever we deploy attack pods across different nodes, we specify a unique label used exclusively by the attack pods and configure an additional *podAntiAffinity* condition based on this label. This prevents unnecessary concentration on the same node and increases the chances of reaching the victim pod.

5 Evaluation

In this section, we evaluate the PodBeater attack. In Section 5.1, we describe our experimental setup. In Section 5.2, we evaluate PodBeater under $\mathcal{T}_u$. In Section 5.3, we evaluate PodBeater under $\mathcal{T}_r$.

5.1 Experimental setup

To evaluate the impact of cluster size on the efficiency of co-location attacks, we conduct experiments across two deployment scales as shown in Table 1: (1) a small-scale cluster with one master node and 20 worker nodes, and (2) a large-scale cluster with one master node and 40 worker nodes. To enable a direct comparison with previous work [5], we adopted the same large-scale configuration and constructed a small-scale cluster with half its size to analyze the impact of scale. The clusters are configured using containerd [3] as the container runtime, and all interactions are performed through the Kubernetes command-line tool `kubectl`. Each node was randomly assigned key-value labels representing hardware characteristics, including CPU type, GPU type, disk type, and memory capacity. Labels are assigned using the command “`kubectl label nodes <nodeName> <key>=<value>`”.

To simulate realistic clusters, we selected 25 of the most widely used services on Docker Hub [4] and deployed them as benign pods. For each pod, labels were randomly assigned based on the Recommended Labels and Example Labels provided by the Kubernetes documentation [8]. The pods are defined using YAML-formatted manifest files and deployed using the command “`kubectl apply -f`

Algorithm 1 PodBeater on threat model $\mathcal{T}_u$.

Require: Label count $L = 9$, maximum attempts N

- 1: Generate label pool $\mathcal{P}$
- 2: Deploy *initial* pods with randomly assigned labels from $\mathcal{P}$
- 3: Deploy *benign* pods with randomly generated affinity conditions
- 4: Select target victim pod $V \in \text{benign}$
- 5: **for** $i \leftarrow 1$ **to** N **do**
- 6: Generate attack pod A_i based on V 's affinity condition
- 7: Deploy A_i
- 8: **if** A_i is placed on the same node as V **then**
- 9: Record number of attempts i
- 10: **Break**
- 11: **end if**
- 12: **end for**

<pod manifest file>”. To evaluate the impact of label diversity, we configured two settings with label counts of 9 and 5, representing high and low diversity, respectively. These labels were selected based on documentation applicable to our environment.

5.2 PodBeater on threat model $\mathcal{T}_u$

In this section, we evaluate the performance of PodBeater in $\mathcal{T}_u$, where the attacker has sufficient knowledge of the victim pod’s scheduling requirement. We evaluate PodBeater with different attack strategies and compare its results against previous work.

Threat model. We assume a strong attacker with sufficient knowledge of the victim pod’s scheduling requirements (i.e., affinity conditions) in the target cluster. Leveraging this knowledge, the attacker can deploy an attack pod to the target cluster by replicating the victim’s affinity conditions. The attacker can also specify multi-value affinity in the pod specification to influence the scheduler’s placement decisions. The scheduler selects a suitable node for each pod solely based on its scheduling requirements. We further assume that the attacker can determine whether co-location with the victim pod has been achieved. This can be verified through side-channel attacks, however, detailed analysis of this technique is beyond the scope of this work.

Experimental design. To conduct a thorough evaluation, we propose two attack strategies and compare their results against REPTTACK. We describe REPTTACK and each strategy as follows:

- **REPTTACK.** The attacker replicates the victim pod’s scheduling requirements and repeatedly deploys attack pods with the same affinity condition.
- **Strategy 1. Deploying an attack pod with an identical multi-value affinity condition.** The attacker extends the victim pod’s affinity condition by appending additional values from the labels of pods likely to be co-located

Table 2. Average number of attack pods required to achieve co-location on $\mathcal{T}_u$

Cluster setting	REPTTACK	PodBeater _{same}	PodBeater _{random}
Small-scale cluster	1.0	1.0	1.0
Large-scale cluster	2.9	2.2	2.1

with the victim. This extended condition remains fixed across all attack attempts. We denote this strategy as *PodBeater_{same}*.

- **Strategy 2. Deploying an attack pod with a distinct multi-value affinity condition.** Similar to Strategy 1, the attacker extends the victim pod’s affinity condition with additional values. However, in each attempt, the attacker randomly selects new values to explore different placement opportunities. We denote this strategy as *PodBeater_{random}*.

We perform our experiment based on Algorithm 1. The experiment begins by initializing the label pool $\mathcal{P}$, where the size of the label pool L is configured to 9 (Line 1 in Algorithm 1). We then deploy initial pods that reference labels from $\mathcal{P}$ but do not specify affinity conditions (Line 2). These initial pods serve as anchors, ensuring that the affinity conditions of subsequently deployed benign pods can be satisfied. This prevents benign pods from remaining in a Pending state due to unsatisfiable affinity conditions. After the initial pods are successfully deployed, the benign pods are deployed with randomly generated affinity conditions (Line 3). Then the attacker selects a victim pod V from benign pods (Line 4). In each trial, the attacker generates an attack pod by replicating the victim pod’s affinity condition and appending additional affinity values inferred from other pods that are likely scheduled on the same node as the victim (Line 6). The attacker then deploys it iteratively (Line 7) until co-location is achieved (Line 8). The number of attack pods required to achieve co-location is recorded (Line 9). The experiment was repeated 10 times under identical settings.

Experimental result. Table 2 presents our experimental results on different cluster scales. In the large-scale cluster, PodBeater demonstrated better performance than the REPTTACK. Specifically, REPTTACK required an average of 2.9 attack pods, while *PodBeater_{same}* and *PodBeater_{random}* required 2.2 and 2.1 pods to achieve co-location with the victim pod, respectively. On the other hand, in the small-scale cluster, all attack strategies achieved co-location with the victim pod by deploying a single attack pod. This performance improvement originates from using node affinity, which effectively narrows down the candidate nodes for placement. Moreover, having fewer nodes and higher pod density in the small-scale cluster makes it more likely that the scheduler places both the victim pod and the attacker’s pod on the same node, even with minimal attempts.

5.3 PodBeater on threat model $\mathcal{T}_r$

In this section, we evaluate the performance of PodBeater in $\mathcal{T}_r$, where the attacker lacks knowledge of the victim pod’s scheduling requirement. We evaluate PodBeater with different attack strategies and discuss their results.

Algorithm 2 PodBeater on threat model $\mathcal{T}_r$.

Require: Label count $L \in \{5, 9\}$, maximum attempts N

- 1: Generate label pool $\mathcal{P}$ of size L
- 2: Deploy *benign* pods with randomly assigned labels from $\mathcal{P}$
- 3: Select target victim pod $V \in \text{benign}$
- 4: **for** $i \leftarrow 1$ **to** N **do**
- 5: Generate attack pod A_i with randomly specified affinity conditions
- 6: Deploy A_i
- 7: **if** A_i is placed on same node as V **then**
- 8: Record number of attempts i
- 9: **Break**
- 10: **end if**
- 11: **end for**

Threat model. We assume an unprivileged attacker who lacks knowledge of the scheduling requirements of any pod in the target cluster. Consequently, the attacker should inevitably deploy attack pods by randomly specifying label values in the affinity condition. All other assumptions are identical to those of $\mathcal{T}_u$.

Experimental design. To conduct a thorough evaluation, we propose three attack strategies and compare their results. We describe each strategy as follows:

- **Strategy 1. Deploying an attack pod with a single value affinity.** The attacker defines an affinity condition with a single affinity value to steer the scheduler’s placement decision. We denote this strategy as $\mathcal{V}_1$.
- **Strategy 2. Deploying an attack pod with three-value affinity.** The attacker defines an affinity condition with three affinity values. We denote this strategy as $\text{PodBeater}_{\mathcal{V}_3}$.
- **Strategy 3. Deploying an attack pod with five-value affinity.** The attacker defines an affinity condition with five affinity values. We denote this strategy as $\text{PodBeater}_{\mathcal{V}_5}$.

We design an experiment based on Algorithm 2. In this experiment, a label pool $\mathcal{P}$ is first generated with a size of label pool $L \in 5, 9$ (Line 1 in Algorithm 2) to reflect cluster diversity by simulating different levels of label diversity. We then deploy benign pods with randomly assigned labels from this pool (Line 2) and select the victim pod V from the benign pods (Line 3). In each trial, the attacker generates attack pod by randomly selecting three or five values from the label pool to construct the affinity conditions (Line 5). This setup allows us to evaluate two types of PodBeater: $\text{PodBeater}_{\mathcal{V}_3}$ and $\text{PodBeater}_{\mathcal{V}_5}$. The attacker deploys the generated attack pod (Line 6) and checks whether it is placed on the same node as the victim (Line 7). The attacker continues to deploy additional attack pods until the co-location is achieved (Line 4-8). The number of attack pods required to achieve co-location is recorded for each trial (Line 9). We repeat the experiment 120 and 240 times for $L = 5$ and $L = 9$, respectively, to ensure reliable results and mitigate scheduling variability.

Experimental result. We evaluate PodBeater with respect to two key factors. First, we vary the total number of labels assigned to benign pods to assess

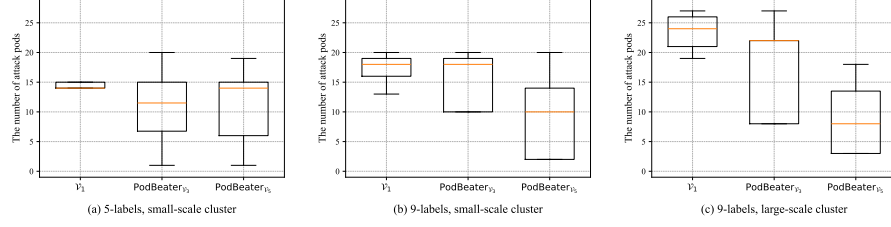


Fig. 3. Distribution of attack pods across attack strategies.

how the size of the label pool influences the attack cost. Second, we configure clusters at two different scales to examine the effect of cluster size on the cost of co-location attacks. Based on these two factors, we determine which of the three proposed attack strategies is the most effective.

Table 3 presents the average number of attack pods required to achieve co-location with the victim pod in the small-scale cluster under two different label counts. The results show that PodBeater consistently reduces the number of required attack pods compared to $\mathcal{V}_1$. Specifically, with a label count of 5, $\mathcal{V}_1$ required an average of 14.44 pods, whereas PodBeater $_{\mathcal{V}_3}$ and PodBeater $_{\mathcal{V}_5}$ reduced this to 10.52 and 10.99, respectively. A similar result is observed with a label count of 9, where both PodBeater $_{\mathcal{V}_3}$ and PodBeater $_{\mathcal{V}_5}$ outperformed $\mathcal{V}_1$. Fig 3(a) and Fig 3(b) show the distribution of required attack pods under the 5-label and 9-label settings, respectively. In the 5-label setting (cf., Fig 3a), PodBeater $_{\mathcal{V}_3}$ is the most efficient, while PodBeater $_{\mathcal{V}_5}$ required slightly more pods. However, in the 9-label setting (cf., Fig 3b), PodBeater $_{\mathcal{V}_5}$ proved to be the most efficient. These results indicate that increasing the number of affinity values does not always improve attack efficiency. To minimize the attack cost, an attacker must carefully tune the number of affinity values based on the cluster characteristics.

Table 4, Fig 3(b), and Fig 3(c) compare the impact of cluster scale. Using 9 labels, we conducted experiments in both small- and large-scale clusters. As the cluster size increased, all strategies required more attack pods to achieve co-location. Specifically, the attack cost increased by approximately 33% for $\mathcal{V}_1$, 15% for PodBeater $_{\mathcal{V}_3}$, and 17% for PodBeater $_{\mathcal{V}_5}$.

In summary, PodBeater consistently reduces attack cost more effectively than $\mathcal{V}_1$ across different label counts and cluster scales. However, using an excessive number of affinity values may diminish attack efficiency. These findings suggest that attackers must adapt affinity configurations based on the characteristics of the target cluster to optimize attack performance.

Table 3. Average number of attack pods required to achieve co-location in the small-scale cluster.

Label count	$\mathcal{V}_1$	PodBeater $_{\mathcal{V}_3}$	PodBeater $_{\mathcal{V}_5}$
5 labels	14.44	10.52	10.99
9 labels	17.57	15.62	8.64

Table 4. Average number of attack pods required to achieve co-location on $\mathcal{T}_r$

Cluster setting	$\mathcal{V}_1$	PodBeater $_{\mathcal{V}_3}$	PodBeater $_{\mathcal{V}_5}$
Small-scale cluster	17.57	15.62	8.64
Large-scale cluster	23.31	18.02	10.14

6 Mitigation

This section presents mitigation approaches to prevent the PodBeater attack.

Enforcing access control over affinity configuration. Kubernetes allows an unprivileged user to specify affinity conditions that influence pod placement. PodBeater exploits this property to efficiently achieve co-location with victim pods. To mitigate this, affinity configurations can be restricted to authorized users, such as specific service accounts, through access control mechanisms. For instance, *PodSecurityAdmission* or *ValidatingAdmissionWebhook* can enforce policies that permit only users in designated RBAC groups to define or modify affinity fields, preventing unauthorized abuse. In addition, integrity checks on affinity conditions can prevent attack pods from being deployed into the cluster.

Detecting and thwarting anomalous behavior. PodBeater achieves co-location by repeatedly launching pods with identical affinity conditions. To mitigate this, the scheduler can monitor affinity usage patterns and restrict repetitive scheduling behavior. For example, the scheduler may delay or reject pods that reuse the same affinity conditions within a short time window. Furthermore, specific affinity conditions can be blocked or restricted through policy to prevent attackers from steering placement toward victim pods.

7 Conclusion

In this paper, we presented *PodBeater*, a co-location attack that exploits Kubernetes’s multi-value affinity feature to achieve efficient co-location. While previous work considers only a threat model in which the attacker has sufficient knowledge of the victim pod referred to as $\mathcal{T}_u$, our work explores both $\mathcal{T}_u$ and a more realistic threat model, $\mathcal{T}_r$, where the attacker lacks such knowledge. In addition, we evaluate PodBeater in both small- and large-scale cluster settings. We first show that, under the $\mathcal{T}_u$, PodBeater requires fewer attack pods than previous work to achieve co-location, demonstrating higher attack efficiency. We then demonstrate that, under the $\mathcal{T}_r$, PodBeater reduces the number of attack pods required to achieve co-location compared to the $\mathcal{V}_1$. However, using too many affinity values can degrade efficiency, suggesting that attackers must tune the number of affinity values based on cluster characteristics. This study provides a systematic analysis of Kubernetes scheduling vulnerabilities under varying attacker models and cluster scales, offering insights to guide the design of more secure orchestration systems.

Acknowledgements. This research was supported by a National Research Foundation of Korea (NRF) grant, funded by the Korean government (MSIT)

(RS-2023-NR077166, RS-2023-00227165). This research was supported by the Korea University Grant.

References

1. Atya, A.O.F., Qian, Z., Krishnamurthy, S.V., La Porta, T., McDaniel, P., Marvel, L.M.: Catch me if you can: A closer look at malicious co-residency on the cloud. *IEEE/ACM Transactions on Networking* **27**(2), 560–576 (2019). <https://doi.org/10.1109/TNET.2019.2891528>
2. Chen, Z., Jiang, F., Chen, B., Li, Y., Zhang, Y., Huang, C., Yang, R., Jiang, F., Chen, J., Xiang, W., et al.: Resource allocation with service affinity in large-scale cloud environments. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE). pp. 5280–5293. IEEE (2024)
3. Containerd Authors: Containerd: An industry-standard container runtime (2025), <https://containerd.io/>, accessed: 2025-05-14
4. Docker Inc.: Docker hub (2025), <https://hub.docker.com/>, accessed: 2025-05-14
5. Fang, C., Wang, H., Nazari, N., Omid, B., Sasan, A., Khasawneh, K.N., Rafati-rad, S., Homayoun, H.: REPTTACK: Exploiting Cloud Schedulers to Guide Co-Location Attacks. In: Proceedings of the Network and Distributed System Security Symposium (NDSS) (2022). <https://doi.org/10.14722/ndss.2022.23149>
6. Han, Y., Chan, J., Alpcan, T., Leckie, C.: Using virtual machine allocation policies to defend against co-resident attacks in cloud computing. *IEEE Transactions on Dependable and Secure Computing* **14**(1), 95–108 (2015)
7. Kubernetes Authors: Kubernetes: Production-grade container orchestration (2025), <https://kubernetes.io>, accessed: 2025-05-10
8. Kubernetes Authors: Objects in kubernetes - labels (2025), <https://kubernetes.io/docs/concepts/overview/working-with-objects/>, accessed: 2025-05-10
9. Liu, F., Yarom, Y., Ge, Q., Heiser, G., Lee, R.B.: Last-level cache side-channel attacks are practical. In: 2015 IEEE symposium on security and privacy. pp. 605–622. IEEE (2015). <https://doi.org/10.1109/SP.2015.43>
10. Makrani, H.M., Sayadi, H., Nazari, N., Khasawneh, K.N., Sasan, A., Rafati-rad, S., Homayoun, H.: Cloak & co-locate: Adversarial railroading of resource sharing-based attacks on the cloud. In: 2021 International Symposium on Secure and Private Execution Environment Design (SEED). pp. 1–13. IEEE (2021)
11. Moreno-Vozmediano, R., Montero, R.S., Huedo, E., Llorente, I.M.: Orchestrating the deployment of high availability services on multi-zone and multi-cloud scenarios. *Journal of Grid Computing* **16**, 39–53 (2018)
12. Pachorkar, N., Ingle, R.: Affinity aware vm colocation mechanism for cloud. *International Journal of Advanced Computer Research* **4**(4), 956 (2014)
13. Varadarajan, V., Zhang, Y., Ristenpart, T., Swift, M.: A placement vulnerability study in multi-tenant public clouds. In: 24th USENIX Security Symposium (USENIX Security 15). pp. 913–928 (2015)
14. Zhang, T., Zhang, Y., Lee, R.B.: Dos attacks on your memory in cloud. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security. pp. 253–265 (2017). <https://doi.org/10.1145/3052973.3052978>
15. Zhao, Z.N., Morrison, A., Fletcher, C.W., Torrellas, J.: Everywhere all at once: Co-location attacks on public cloud faas. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. pp. 133–149 (2024). <https://doi.org/10.1145/3617232.3624867>